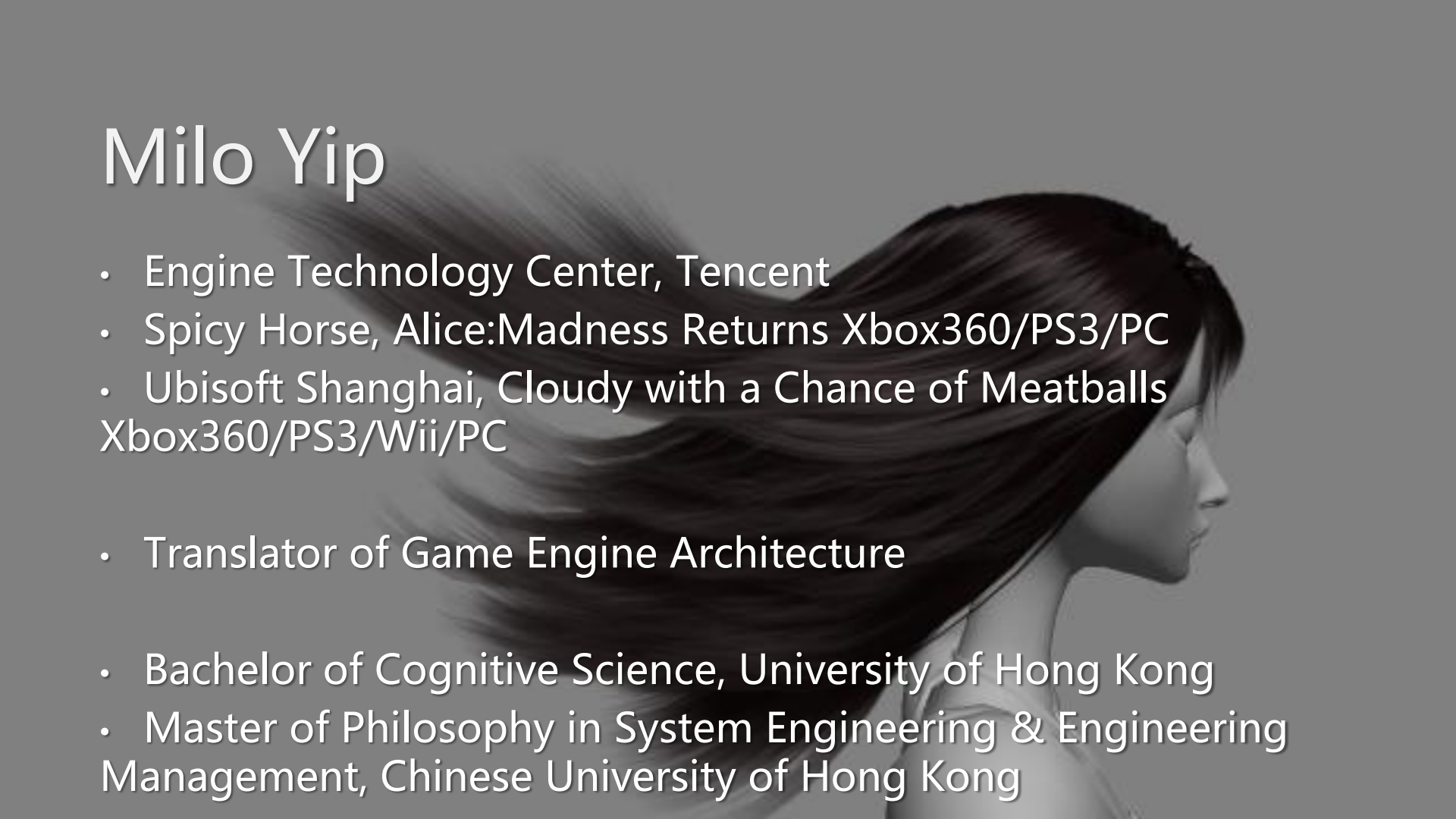
A Data-Oriented Programming Paradigm for Optimal Performance

Milo Yip
Expert Engineer, Tencent



GAME DEVELOPERS CONFERENCE™ CHINA
SHANGHAI INTERNATIONAL CONVENTION CENTER
SHANGHAI, CHINA · OCTOBER 19–21, 2014

Milo Yip



- Engine Technology Center, Tencent
- Spicy Horse, Alice:Madness Returns Xbox360/PS3/PC
- Ubisoft Shanghai, Cloudy with a Chance of Meatballs Xbox360/PS3/Wii/PC

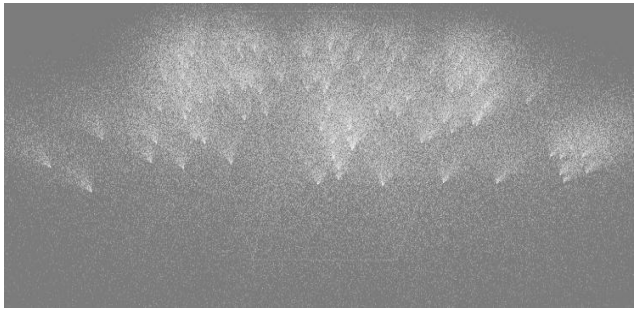
- Translator of Game Engine Architecture

- Bachelor of Cognitive Science, University of Hong Kong
- Master of Philosophy in System Engineering & Engineering Management, Chinese University of Hong Kong

Performance Test

- i7 2.93Ghz + GTX 460
- 100 instances × 10000 particles = **1M particles**

	Unity 4.3	TAG Prestige
Memory	58MB	27MB
FPS	4 FPS (CPU bound)	80 FPS (GPU bound)
CPU Simulation	250ms	6.2ms
CPU Render	44ms	5.7ms



CONTENTS

· Object-oriented vs Data-oriented
· AOS, SOA and Varieties
· Dynamic struct
· Practical Uses
· Summary

CONTENTS

- **Object-oriented vs Data-oriented**
- AOS, SOA and Varieties
- Dynamic struct
- Practical Uses
- Summary

History of C++

- 1972: C
- 1979: C with class
- 1983: C++
- 1998 to now: Standardized C++98, C++03, C++11, C++14

Common Programming Paradigms in C++

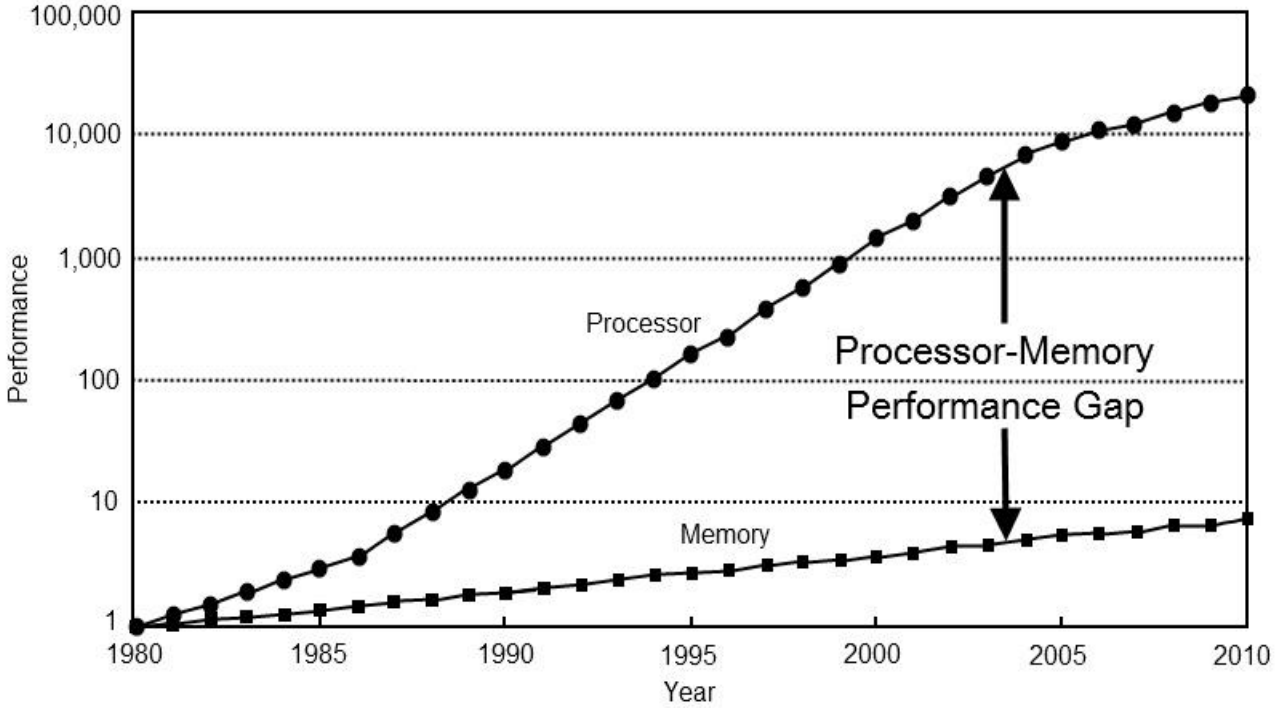
- Procedural
- Object-Oriented
- Meta-Programming
- Functional

OOP

- Combines data structure and methods as object
- Groups objects with common behavior as class
- Encapsulates internal details in class
- Specialization/generialization via inheritance
- Polymorphism via dynamic dispatching with object type

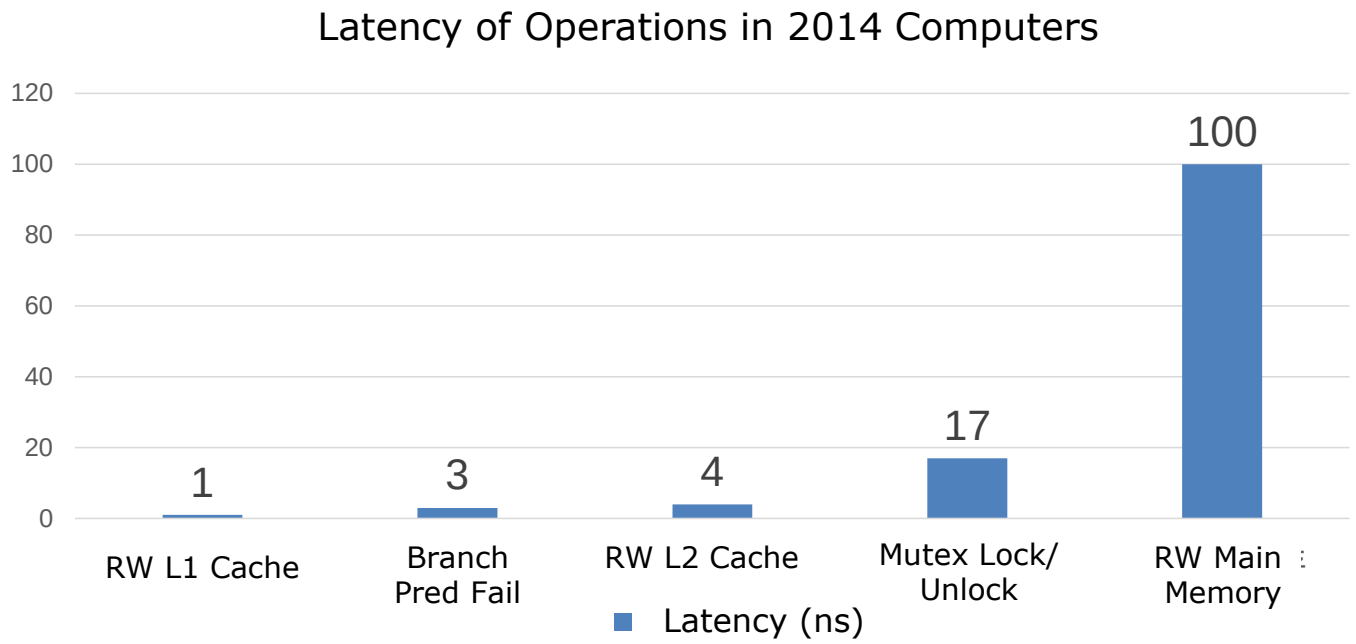
- Isn’ t OOP great? But…

Hardware Bottleneck Shift



Hennessy et. al, **Computer architecture: a quantitative approach**

Latency Issue

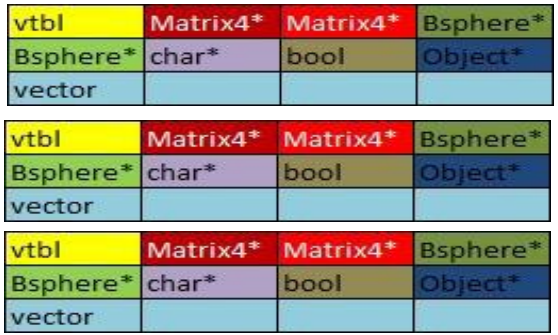


Latency Numbers Every Programmer Should Know http://www.eecs.berkeley.edu/~rcs/research/interactive_latency.html

OOP may be not cache-friendly

- Due to **encapsulation**, data are packed together. e.g. from [1]

```
class Object
{
    // <methods removed for clarity>
    Matrix4 *m_Transform;
    Matrix4 *m_WorldTransform;
    BoundingSphere *m_BoundingSphere;
    BoundingSphere *m_WorldBoundingSphere;
    char* m_Name;
    bool m_Dirty;
    Object* m_Parent;
};
```



- When each iteration just access a few member variables in inner-loop, it wastes a lot of cache space

DOP

- Data-Oriented Programming
  - discussion in game since around 2009
  - PS3 and others encounter perf. issues related to OOP
- Main considerations in DOP
  - Data layout
  - Access pattern of data
  - Improves cache usage to gain much better performance.

Applicable areas for DOP

- Suitable for
  - Processing of large amount of homogeneous data
  - Few branching
- Applications in Games
  - Particles, Soft-body, Rigid-body, Fluid Simulation
  - Collision, Visibility Detection
  - Skeletal Animation
  - Group Behavior Simulation

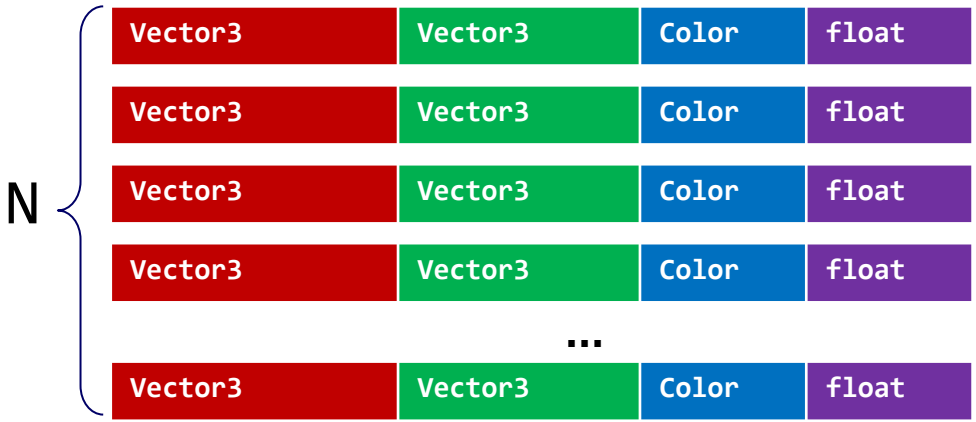
CONTENTS

- Object-oriented vs Data-oriented
- **AOS, SOA and Varieties**
- Dynamic struct
- Practical Uses
- Summary

Array of Struct (AOS)

- The most common data layout in C++
- E.g., in particle systems, each particle is a struct

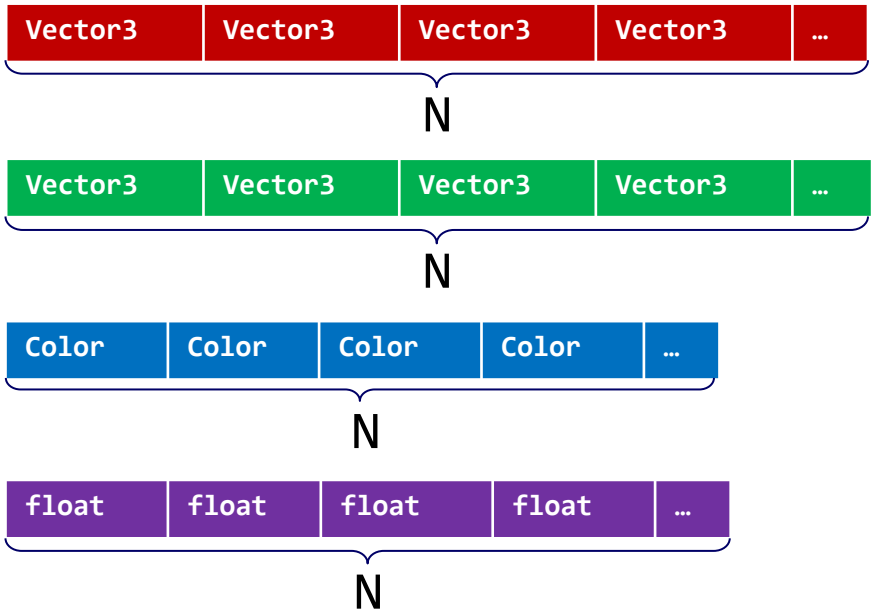
```
struct Particle {
    Vector3 position;
    Vector3 velocity;
    Color color;
    float age;
    // …
} particles[N];
```



Struct of Array (SOA)

- SOA stores homogeneous data continuously in arrays

```
struct Particles {
    Vector3 position[N];
    Vector3 velocity[N];
    Color color[N];
    float age[N];
    // …
}particles;
```



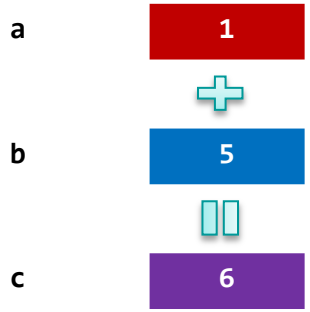
Often SOA > AOS

- Cache-friendly
  - SOA does not need to add padding for alignment
  - Many operations only require a few fields
- Save memory
  - No waste on padding, perfect alignment
- High performance
  - Can use SIMD to read/write memory fast

SIMD (Single Instruction Multiple Data)

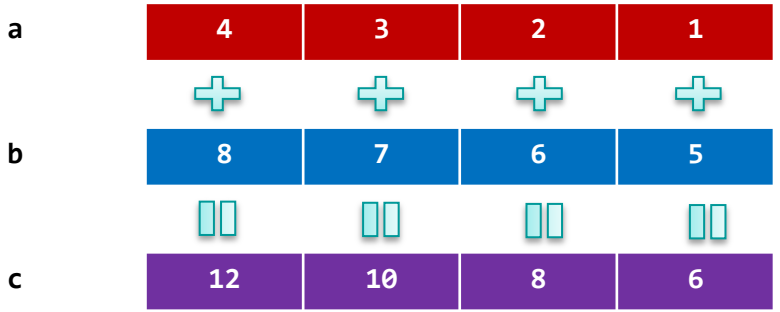
General CPU instructions are SISD (Single Instruction, Single Data)

```
float a = 1;
float b = 5;
float c = a + b;
```



SIMD instruction operates on multiple data in parallel

```
__m128 a = _mm_setr_ps(1, 2, 3, 4);
__m128 b = _mm_setr_ps(5, 6, 7, 8);
__m128 c = _mm_add_ps(a, b);
```



SOA is more suitable for SIMD

- SOA can fully utilize SIMD computation throughput (best solution)
- E.g., a 3D dot product

$$\vec{a} \cdot \vec{b} = a_x b_x + a_y b_y + a_z b_z$$

Comparison for dot-product (4-way SIMD)

	AOS	AOS & SIMD	SOA & SIMD

Pseudo-code	`x = a.x * b.x;`
	`y = a.y * b.y;`
	`z = a.z * b.z;`
	`dot = x + y + z;`
		`m = a * b;`
		`dot = m + m.yyyy + m.zzzz;`
			`x = ax * bx`
			`y = ay * by`
			`z = az * az`
			`dot = x + y + z`

Time	12 mul + 8 add	4 mul + 8 add + 2 swz	12 mul + 8 add
Dot-product	1	1	4
Through-put	12 mul + 8 add	4 mul + 8 add + 2 swz	3 mul + 2 add

- When computing length, normalization, SOA & SIMD saves a lot!

Practical Examples

- Simulates linear motion of n particles using Euler integration:

$$\vec{v}_i(t + \Delta t) = \vec{v_i}(t) + \vec{a}\Delta t$$

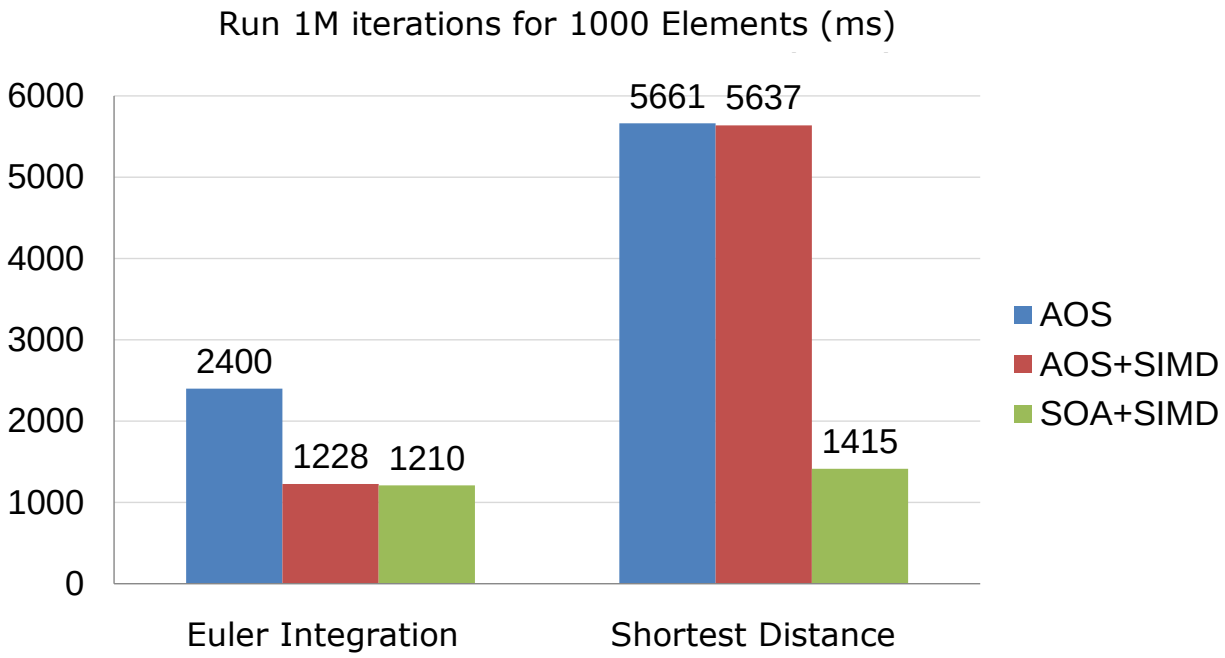
$$\vec{r_i}(t + \Delta t) = \vec{r_i}(t) + \vec{v_i}(t)\Delta t$$

- Computes the shortest distance between a point $\vec{p}$ and n spheres:

$$d_i = \|\vec{p} - \vec{c}_i\| - r_i$$

$$= \sqrt{(\vec{p} - \vec{c}_i) \cdot (\vec{p} - \vec{c}_i)} - r_i$$

Performance Comparison



CONTENTS

- 面向对象 vs 面向数据
- AOS、SOA及变种
- **动态struct**
- 实际应用
- 总结

CONTENTS

- Object-oriented vs Data-oriented
- AOS, SOA and Varieties
- **Dynamic struct**
- Practical Uses
- Summary

Problem of "Dynamic struct"

- How to create a struct, of which the fields are specified during runtime?
  - Also the data types of fields
- For each type of struct, it needs to create a lot of instances.
  - E.g., vertices of mesh, particles of same type, enemy NPCs of same type
- C++ are statically typed language
  - It can only define each struct in compile-time

Solution 1：Put in Everything

- All fields that may be used are put into the struct.

  Waste memory, cache-unfriendly

- After adding new features, the original data also cost more memory.

  Incerasing resistance to add new features

A Real-life Example of a Particle System

```
class Particle {
    // ...
    float m_fLifespan;
    float m_fAge;
    float m_fRotation;
    float m_fRotationSpeed;
    float m_fScalar;
    int m_nImageIndex;
    D3DXVECTOR3 m_vPosition;
    D3DXVECTOR3 m_vLastPosition;
    D3DXVECTOR3 m_vVelocity;
    D3DXVECTOR3 m_vNormal;
    D3DXVECTOR3 m_uAxis;
    D3DXVECTOR3 m_vAxis;
    DWORD m_dwFixedColor;
    FxObject *m_pAttachedFxObject;
};
```

Solution 2 : Using union

```
struct S2 {
    union {
        Vector3 v;
        float f;
    }a;
    union {
        // ...
    };
};
```

- Need to determine which parts will not be used at the same time
- Also waste on padding

Solution 3: Key-Variant Table

```
struct S3 {
    unordered_map<Key, Variant> kv;
};
```

- Very flexible, but
  - Require a lot of struct instances.
  - Each instance has overhead of a map.
  - All Variants have overheads.

Solution 4: Flexible Table

- Uses the concept of table (relation) in database

  table: row × column → cell

- Define meta-information of each table during runtime
  - column name, type, default value

- Can also define types during runtime
  - Name, size, alignment of a type

AOS Table: Example Usage

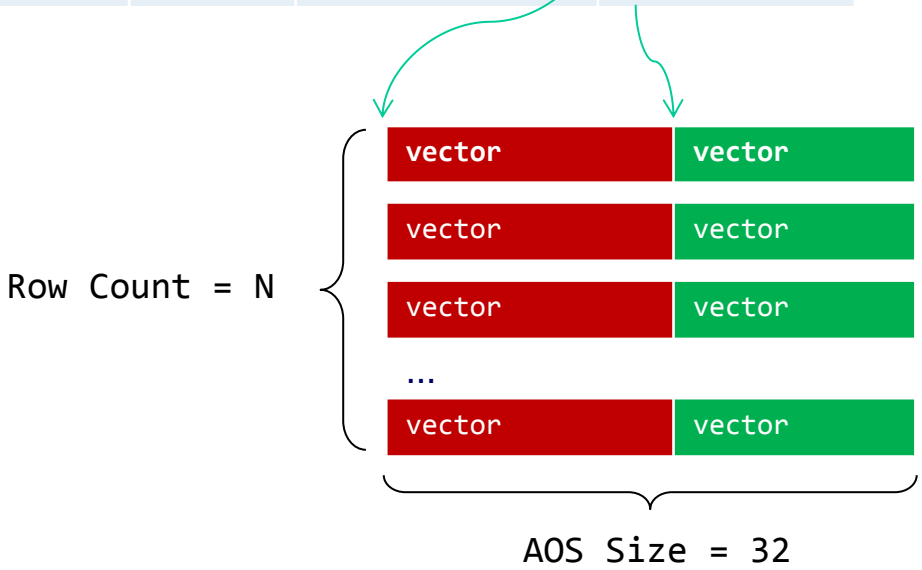
```
MetaTable meta;
TypeID vectorType = meta.AddType("vector", 16, 16);
AttributeID positionAttribute = meta.AddAttribute("position", vectorType, _mm_setzero_ps());
AttributeID velocityAttribute = meta.AddAttribute("velocity", vectorType, _mm_setzero_ps());

AOSTable particles(meta);
particles.ReserveRows(N);
particles.AppendRows(N);

for (size_t row = 0; row < N; row++) {
    particles.SetValue(row, positionAttribute, _mm_setr_ps(...));
    particles.SetValue(row, velocityAttribute, _mm_setr_ps(...));
}
```

AOS Table: Memory Layout

AttributeID	Name	Type	Default	AOS Offset
0	“position”	vector	{0, 0, 0, 0}	0
1	“velocity”	vector	{0, 0, 0, 0}	16



AOS Table: Iterating

```
__m128* p = particles.GetValueRaw<__m128>(0, positionAttribute));
__m128* v = particles.GetValueRaw<__m128>(0, velocityAttribute));
const size_t stride = particles.GetAOSSize();

for (size_t i = 0; i < N; i++) {
    *p = _mm_add_ps(*p, _mm_mul_ps(*v, dt));  // p += v * dt
    *v = _mm_add_ps(*v, adt);                 // v += a * dt
    p = (__m128*)((char*)p + stride);
    v = (__m128*)((char*)v + stride);
}
```

SOA Table: Example Usage

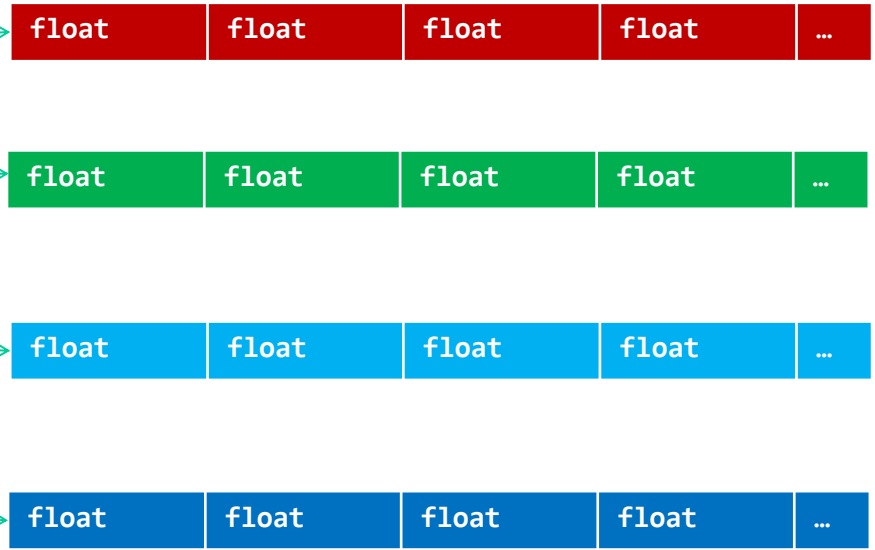
```
MetaTable meta;
const TypeID floatType = meta.AddType("float", 4, 16);
const AttributeID positionXAttribute = meta.AddAttribute("positionX", floatType, 0.0f);
const AttributeID positionYAttribute = meta.AddAttribute("positionY", floatType, 0.0f);
const AttributeID positionZAttribute = meta.AddAttribute("positionZ", floatType, 0.0f);
const AttributeID velocityXAttribute = meta.AddAttribute("velocityX", floatType, 0.0f);
 // ...

SOATable particles(meta);
particles.ReserveRows(N);
particles.AppendRows(N);

for (size_t i = 0; i < N; i++) {
    particles.SetValue(i, positionXAttribute, ...);
    // ...
}
```

SOA Table: Memory Layout

AttributeID	Name	Type
0	“positionX”	float
1	“positionY”	float
2	“positionZ”	float
3	“velocityX”	float
4	“velocityY”	float
5	“velocityZ”	float



…

SOA Table: Iterating

```
__m128* px = particles.GetValueRaw<__m128>(0, positionXAttribute);
__m128* py = particles.GetValueRaw<__m128>(0, positionYAttribute);
__m128* pz = particles.GetValueRaw<__m128>(0, positionZAttribute);
__m128* vx = particles.GetValueRaw<__m128>(0, velocityXAttribute);
__m128* vy = particles.GetValueRaw<__m128>(0, velocityYAttribute);
__m128* vz = particles.GetValueRaw<__m128>(0, velocityZAttribute);

for (size_t i = 0; i < N / 4; i++) {
    px[i] = _mm_add_ps(px[i], _mm_mul_ps(vx[i], dt)); // p += v * dt
    py[i] = _mm_add_ps(py[i], _mm_mul_ps(vy[i], dt));
    pz[i] = _mm_add_ps(pz[i], _mm_mul_ps(vz[i], dt));
    vx[i] = _mm_add_ps(vx[i], axdt); // v += a * dt
    vy[i] = _mm_add_ps(vy[i], aydt);
    vz[i] = _mm_add_ps(vz[i], azdt);
}
```

Flexible Table: Pros

- Define fields/types during runtime
- Supports AOS/SOA
- Decouple between Program and Data
  - Each module can use string to obtain AttributeID
  - Modules are compiled independently, and dynamically bind during runtime

CONTENTS

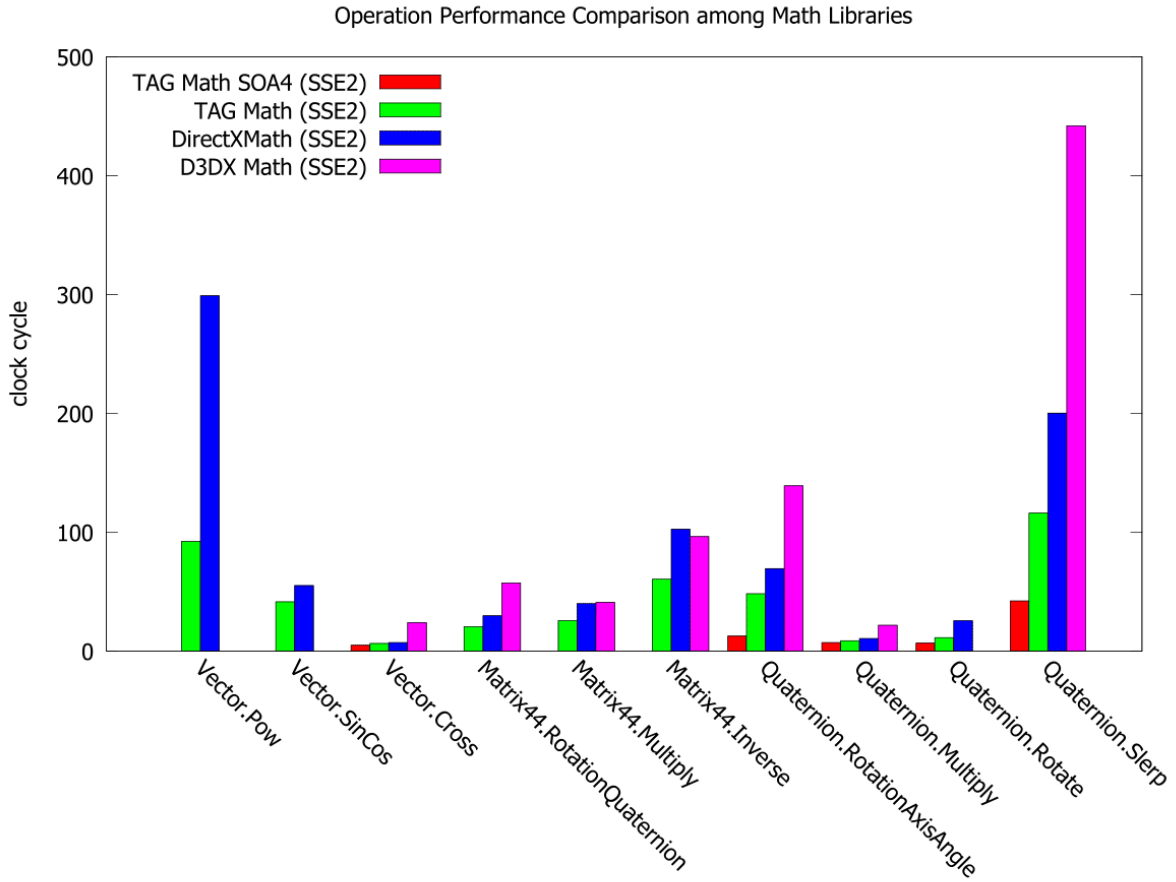
- Object-oriented vs Data-oriented
- AOS, SOA and Varieties
- Dynamic struct
- **Practical Uses**
- Summary

TAG Math

- A Math Library with SIMD acceleration
  - Supports Intel SSE2/3/4, ARM NEON
- Provide AOS/SOA operations:

```
c = Vector3Dot(a, b);
c = Vector3SOA4Dot(ax, ay, az, bx, by, bz);
```

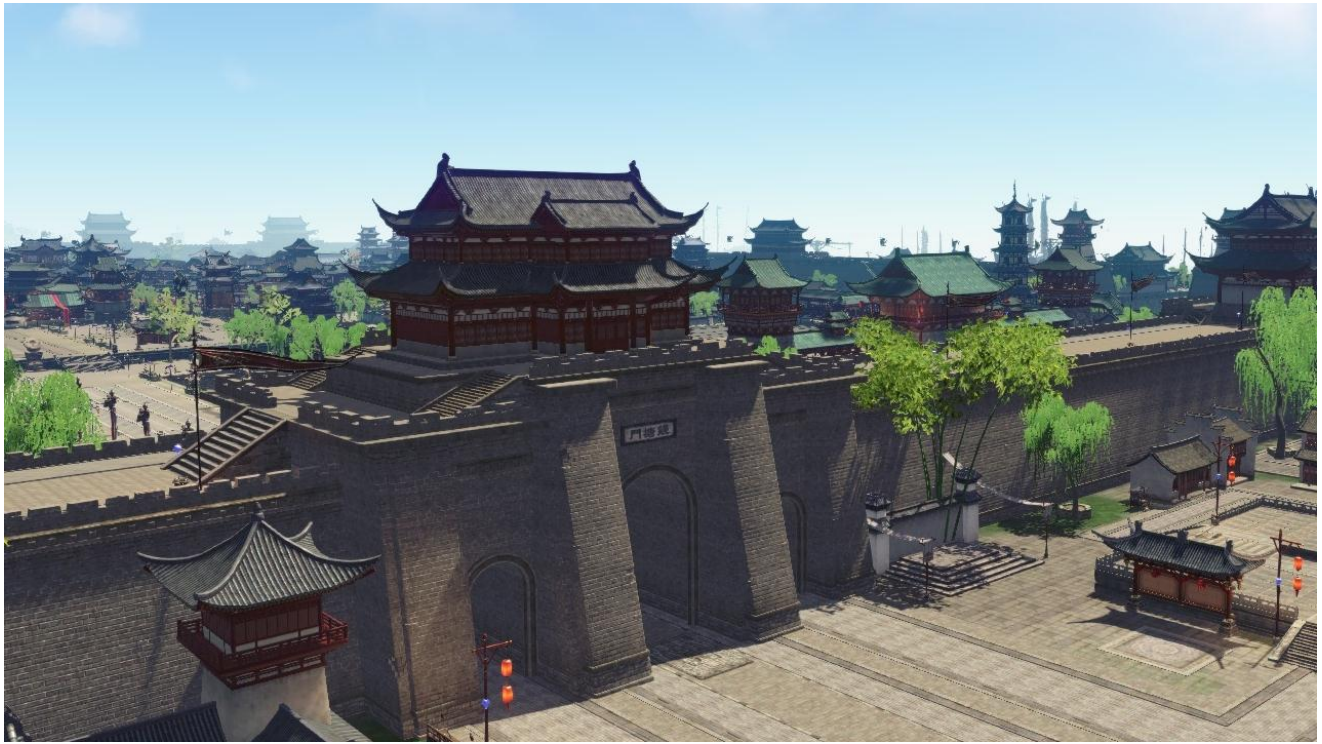
TAG Math Performance Comparison



TAG Visibility

- A Visibility Determination Solution for 3D scenes, ref to [2][3]
  - View Frustum Culling
  - Occlusion Culling
  - Contribution Culling
- Use AOS layout to store Bounding Volumes
- Only 2 layers of Loose Grid instead of Octree kind of structure
  - Fully dynamic scene management
  - Continuous memory access, homogeneous computation

Performance Comparison: 3D Scene Example

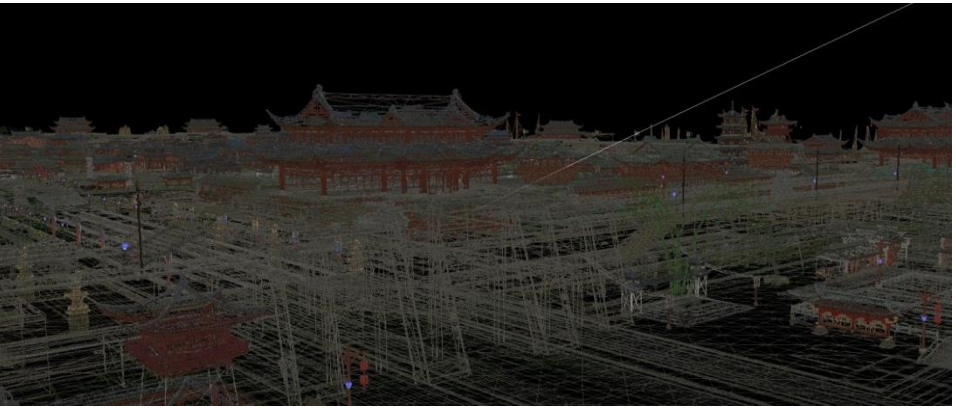


Performance Comparison: The Occlusion



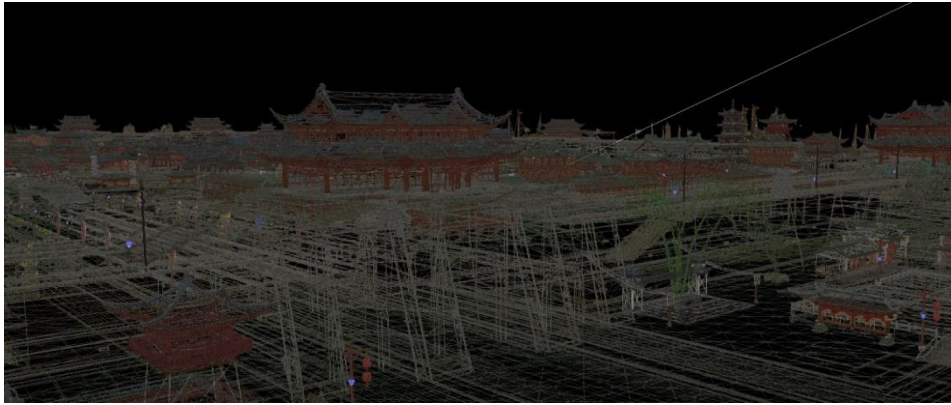
Performance Comparison: Occlusion Culling

Off



- DP: 8050
- Triangles: 7630K

On

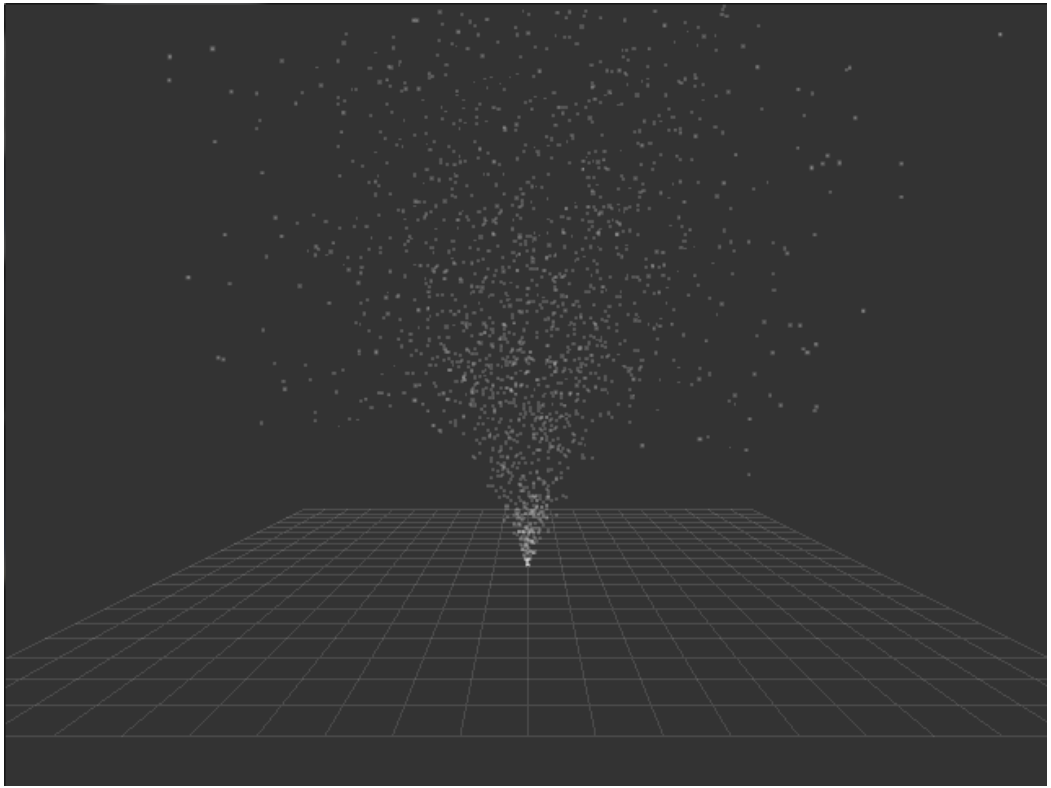


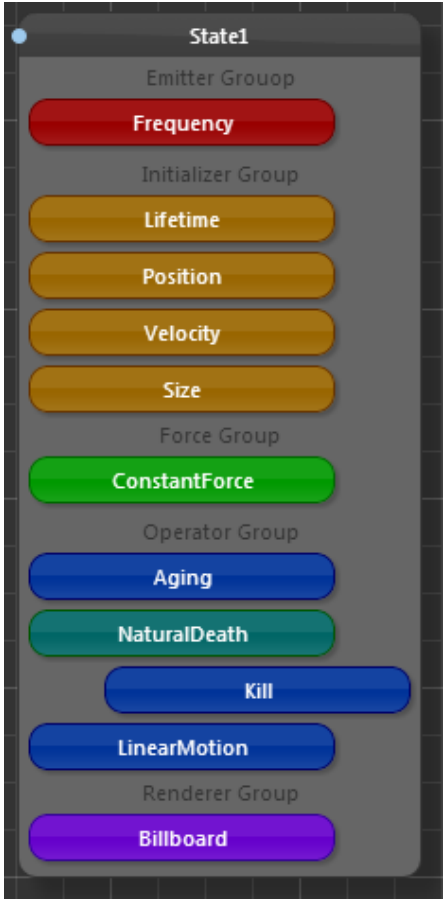
- DP: 4201
- Triangles: 4690K

TAG Prestige

- An Extensible Modular Particle System
  - Use Flexible SOA to store particles
  - Each module specifies which particle attributes it needs
  - Computations are implemented in SOA SIMD
  - High performance, low memory footprint
- Other advanced features in the architecture
  - State transition of particles
  - Nested Particle System
    (each particle can be a particle system)

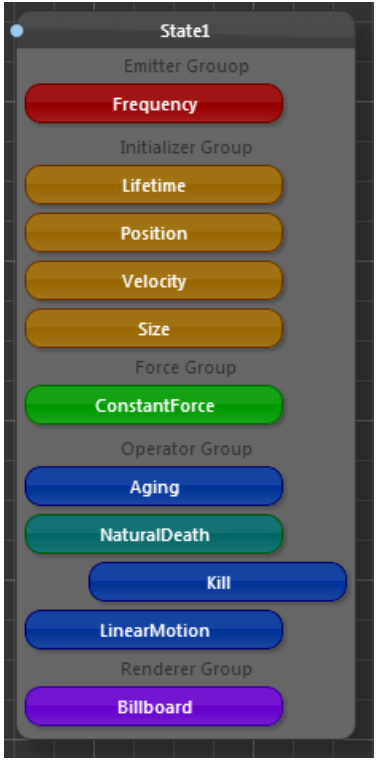
Simple Example





Modules Define Particle Attribute Set

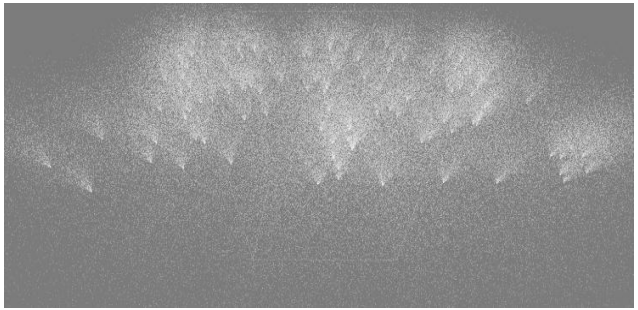
Module	Required Attributes (those inside brackets are temp var)
FrequencyEmitter	-
LifetimeInitializer	Lifetime
PositionInitializer	PositionX/Y/Z
VelocityInitializer	VelocityX/Y/Z
SizeInitializer	Size
ConstantForce	(ForceX/Y/Z)
AgingOperator	Age
NaturalDeathTest	Age, Lifetime
KillOperator	-
LinearMotionOperator	PositionX/Y/Z, VelocityX/Y/Z, (ForceX/Y/Z)
BillboardRenderer	PositionX/Y/Z, Size
All attributes needed for this state	PositionX/Y/Z, VelocityX/Y/Z, (ForceX/Y/Z), Age, Lifetime, Size



Performance Test

- i7 2.93Ghz + GTX 460
- 100 instances × 10000 particles = **1M particles**

	Unity 4.3	TAG Prestige
Memory	58MB	27MB
FPS	4 FPS (CPU bound)	80 FPS (GPU bound)
CPU Simulation	250ms	6.2ms
CPU Render	44ms	5.7ms



TAG Velvet

- A Soft-body Simulator for Games
- Using Flexible SOA to Store Attributes
  - Attributes of nodes and links
- Advanced Features
  - Long Range Attachment (LRA) [4]
  - Shape Matching [5]
  - Continuous Collision Detection (CCD)

Demonstration

- Using the freely available UnityChan, without modification
http://unity-chan.com/
- Use Velvet to simulate
  - Hair strands above forehead
  - 2 braids
  - Head ribbons



Performance Data

- Bones : 30
- Colliders : 4

Hardware	Time (ms)
i7@2.93Ghz	0.2
iPhone 4S	0.9
iPad 4	0.6
Nexus 10	0.4


default@unitychan
smile1@unitychan
smile2@unitychan
conf@unitychan
sap@unitychan
angry1@unitychan
angry2@unitychan
eye_close@unitychan
0.20 ms
VelvetEnvironmentHair
Gravity
Wind
Damping
VelvetEnvironmentRibbon
Gravity
Wind
Damping
TAGVelvet

Other Potential Applications

- **Game Logic**: Component-based, Attribute-based Object Model
- **AI**：Many NPC with same behavior (e.g. Flocking)
- **Animation**: Sampling, Blending, Hierarchy Transformation
- **Physics**: Intersection/Collision Detection, Rigid Body, Soft Body, Fluid

CONTENTS

- Object-oriented vs Data-oriented
- AOS, SOA and Varieties
- Dynamic struct
- Practical Uses
- **Summary**

Data-Oriented Programming (DOP)

- Compiler can optimize code of execution
  - But almost cannot optimize data layout at all!
  - Data became the current and future bottleneck
- DOP objective: To solve some performance/memory problems of OOP
- DOP how-to: consider about data layout and access pattern

SOA vs AOS

SOA: Pros

- Cache-Friendly
- Optimal SIMD throughput
- Saves memory
  (saves paddings, perfect alignment)

SOA: Cons

- May need AOS layout in external system, e.g. VB needs conversion
- Branching makes waste
- Consective elements cannot depend on each other
- Need special treatment for last remaining elements

Flexible Table

- A solution for dynamic struct during runtime
- Ease to use
- High Performance
  - Almost no overhead to static struct
- Contain both AOS/SOA implementation
- Reference implementation https://github.com/miloyip/flexible

For thinking

"We should forget about small efficiencies, say about 97% of the time: premature optimization is the root of all evil"

Donald Knuth, 1974

- DOP needs to be introduced in design stage, and affects most parts of implementation

References

1. ALBRECHT, "Pitfalls of Object Oriented Programming", *GCAP Australia*, 2009. http://research.scee.net/files/presentations/gcapaustralia09/Pitfalls_of_Object_Oriented_Programming_GCAP_09.pdf
2. COLLIN, "Culling the Battlefield", GDC 2011. http://dice.se/wp-content/uploads/CullingTheBattlefield.pdf
3. HILL, COLLIN. "Practical, dynamic visibility for games", *GPU Pro 2*, 2011.
4. KIM, CHENTANEZ, MÜLLER, "Long range attachments: a method to simulate inextensible clothing in computer games." *Proceedings of the 11th ACM SIGGRAPH/Eurographics conference on Computer Animation*. Eurographics Association, 2012. http://www.matthiasmueller.info/publications/sca2012cloth.pdf
5. MÜLLER, HEIDELBERGER, TESCHNER, GROSS, "Meshless Deformations Based on Shape Matching", in *Proceedings of SIGGRAPH'05*, pp 471-478, Los Angeles, USA, July 31 - August 4, 2005. http://www.matthiasmueller.info/publications/MeshlessDeformations_SIG05.pdf
6. ACTION, "Data-Oriented Design and C++", cppcon 2014. https://github.com/CppCon/CppCon2014/tree/master/Presentations/Data-Oriented%20Design%20and%20C%2B%2B